The problem

contacts-v1 documents are **append-only**. Every `<contact> <pX> <pY>` asserts a contact, absence means "no contact", and there is no way to take one back. So a rollout that commits to a wrong contact early carries that mistake to the end, and it costs precision for every statement that follows.

Issue #142 found that apparent under-generation is a difficulty symptom rather than a decoding bug — which points at the model's inability to *revise*, not its stopping rule.

The idea: let documents retract

Issue #158 added one token. `<retract> <pX> <pY>` takes back a previously emitted contact, turning the structure section from an unordered set into an ordered **edit list** whose answer is whatever is still live at `<end>`.

Appended last in the vocab, so every pre-existing token id is unchanged and existing checkpoints just grow their embedding by one row.

To train a model to use it we need documents that *demonstrate* self-correction. That is this experiment.

Where the decoys come from

Not synthetic negatives — the **base model's own false positives**. They are exactly the mistakes it is tempted to make, so the context leading to each one is on-distribution in a way an invented negative never is.

The key design choice: re-condition, don't splice

The obvious approach — sample a rollout, then splice `<retract>` in after each wrong contact — is wrong. It leaves the entire tail of the document conditioned on a context that still contains the mistake and never contains the correction.

Instead we generate **with the model in the loop**: after every retraction the model's prompt is rebuilt from the currently-**live** contacts and generation continues. Two streams run in lockstep — the output document (which carries the `<retract>` tokens) and a clean conditioning prompt (which never does, because the base model has never seen one).

Every contact stays on-distribution. The only synthetic tokens in the corpus are the retract statements themselves.

When to retract: the model's own collapsing posterior

Timing must be a signal the trained model can recompute from its own context, or we teach it to retract on information it will not have.

So we score each queued contact with the base model's leave-one-out belief `s(c)` against the *committed* set, and retract when that belief collapses (relative to its peak, below a floor, or out of the top-R). A false positive becomes inconsistent with the fold as true contacts accumulate; a true contact does not.

Ground truth never decides timing — only that the final set is correct.

Correctness is structural

A budget-reserved flush guarantees the invariant: the loop only continues while it can still afford to retract every live non-GT pair and emit every missing one. So a badly-tuned trigger degrades *realism*, never *correctness*.

Result: **1,023,997 of 1,023,997 documents fold to exactly their ground truth**, verified by re-deriving the fold per document rather than trusting the generator's own bookkeeping.

The corpus

| | | |---|---| | documents | **1,023,997** | | tokens | 1,076,910,057 | | mean contacts / doc | 186.3 | | **mean retracts / doc** | **33.1** | | documents containing a retraction | 79.7% | | mean protein length | 193.7 residues | | truncated | 0 |

Source: the ESMFold2-Atlas distillation set, with ground truth from exp139's saved pyconfind contacts — no pyconfind at generation time.

Does retraction track wrongness?

This is the pass/fail. Across **32,849,569** false positives the base model emitted:

- **76.4%** were retracted by its own collapsing posterior - P(false positive | retracted) = **0.974**, against a base rate of 0.166 - **enrichment 5.85x** — 97% of the achievable ceiling (6.02x) - **0** true contacts were ever retracted by the trigger

Retraction is not noise. It is a sharp, ground-truth-free wrongness detector.

And it is delayed

Mean **17.9** statements between a contact and its retraction (median 9); only 0.1% are immediate.

That spread is the point. An immediately-retracted mistake teaches a trivial local pattern; a mistake retracted 18 statements later teaches the model to revise once the accumulating picture turns against an earlier call — the long-range capability the format exists for.

Making it fast enough

The first working loop cost 14.7 s/document. Profiling put 86% in `propose` at **batch size 1**.

- the engine became a **generator** that yields its model requests, so many proteins advance in lockstep and their proposals batch on the GPU - the decode budget dropped 12 → 6 tokens (a contact statement is 3) - `score` computes only the tails its targets need, not all L (numerically identical — there is an equivalence test)

14.7 → 1.68 s/doc (A5000), **1.09 s/doc** on an H100. An 8.8x change that turned scale from an algorithmic problem into a fan-out problem.

Running it

48 × 1 H100 on CoreWeave `cw-rno2a` at **batch priority**, ~4.5 h, **0 worker failures**.

Two fixes were needed along the way, both worth remembering:

- **write parquet parts every 250 documents, not per shard** — a whole-shard write meant one preemption discarded up to 4,000 documents of GPU work - **`max_task_failures` is a separate field defaulting to 0** — GPU reclamation arrives as a SIGTERM recorded as a *failure*, not a preemption, so `max_retries_preemption` never applied and the fleet bled 48 → 28 workers

Published

`hf://buckets/open-athena/MarinFold/data/document_structures/contacts_v1_backtracking/`

Public, anonymously readable, tokenizer co-located. `<retract>` is the last token of the contacts-v1 vocab; the coordinate/crops superset tokenizers also carry it, so retraction-bearing documents can go into a mixture.

What this does not yet tell us

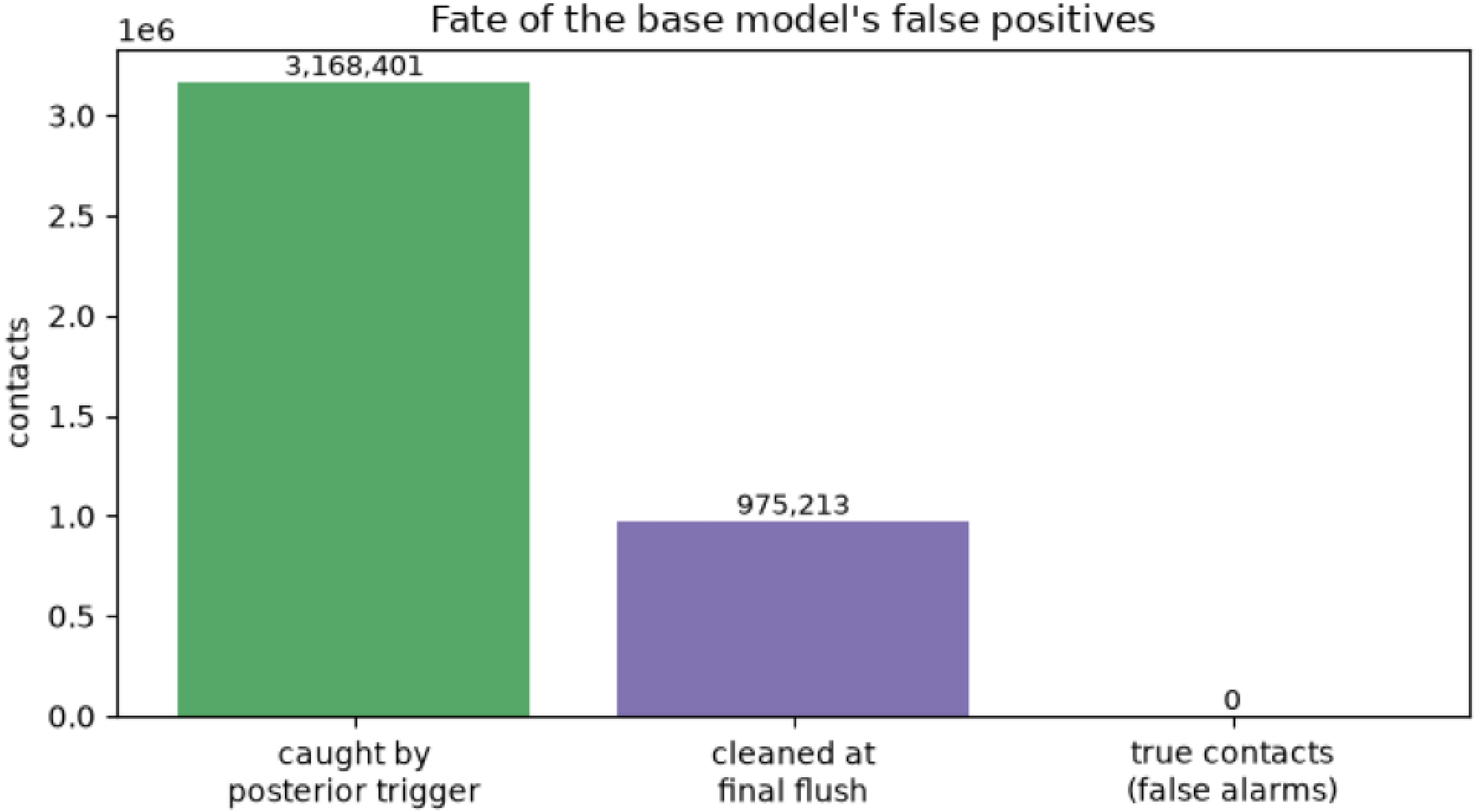
The corpus is *authored* to be discriminative. Whether a **trained** model reproduces that behaviour is issue #160 — and the honest bar is high: #82's resample+vote already recovers much of what a single bad rollout loses, so backtracking has to beat that, not greedy decoding.

The diagnostics for it are built and validated (10 unit tests; enrichment, retract rate, retraction distance, recovery). Note the corpus's **recall = 1.000 is an artifact of the correctness flush** — a trained model has no flush, so precision and enrichment are the numbers that transfer.

The 10% single-retract "probe" class is deliberately deferred.

fp_catch.png

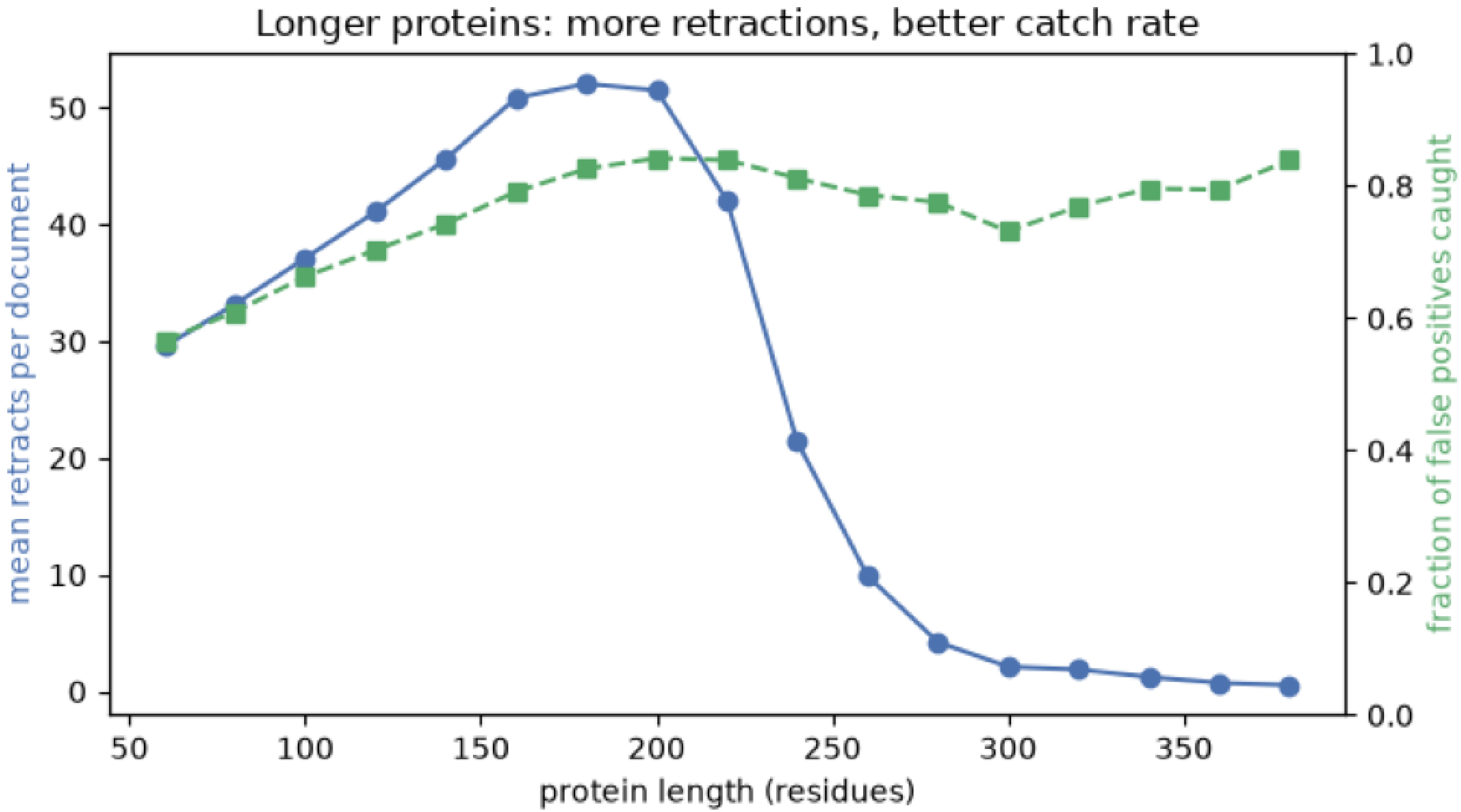
Of 4,143,614 false positives the base model emitted, 76.5% were retracted by its own collapsing posterior; the rest are removed by the correctness flush. The trigger wrongly retracted 0 true contacts.



`$ make_plots.py --shards 2`

length_relationship.png

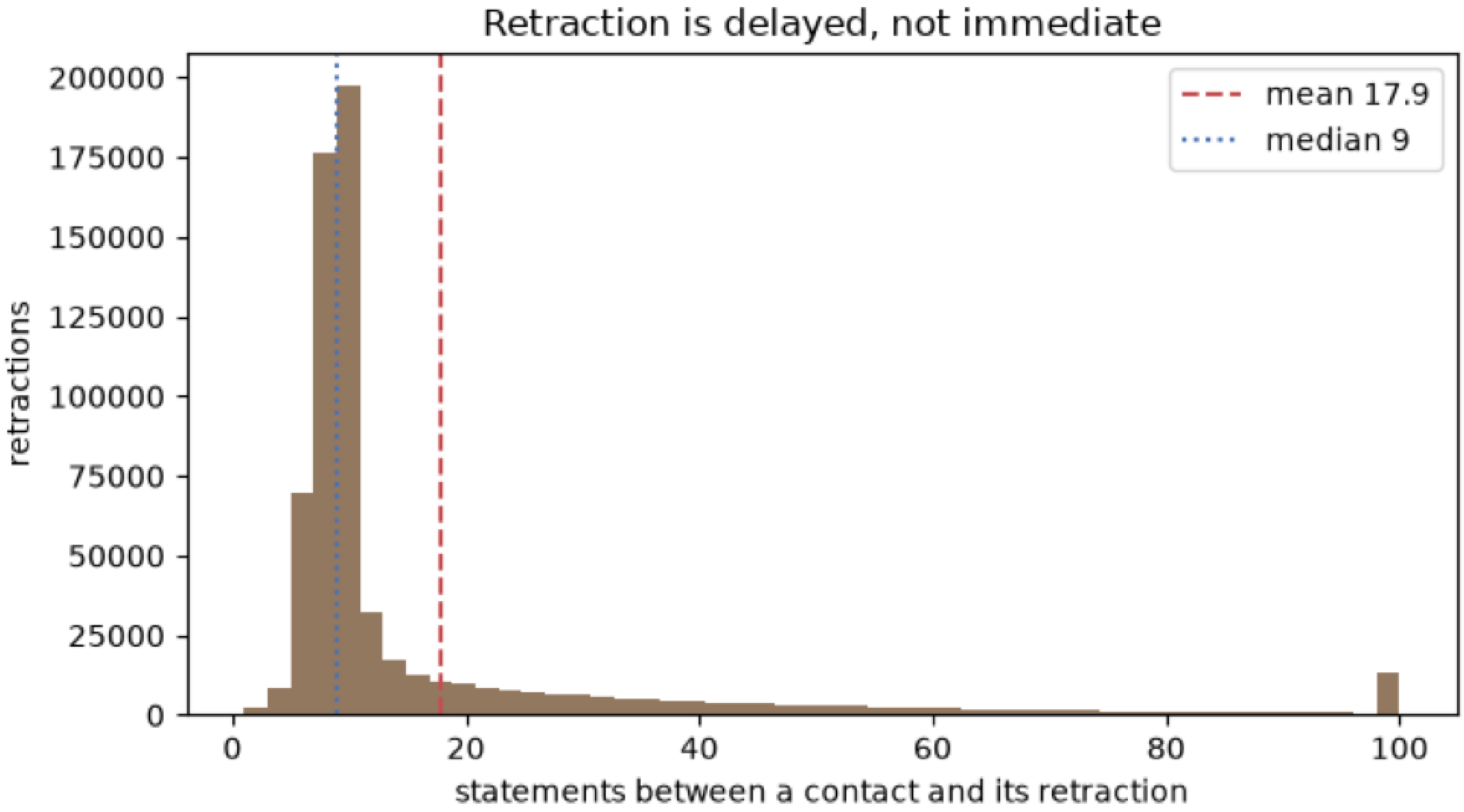
Retractions per document rise with protein length, and so does the fraction of false positives the trigger catches — a bigger committed set gives the posterior more to lean on.



`$ make_plots.py --shards 2`

retract_distance.png

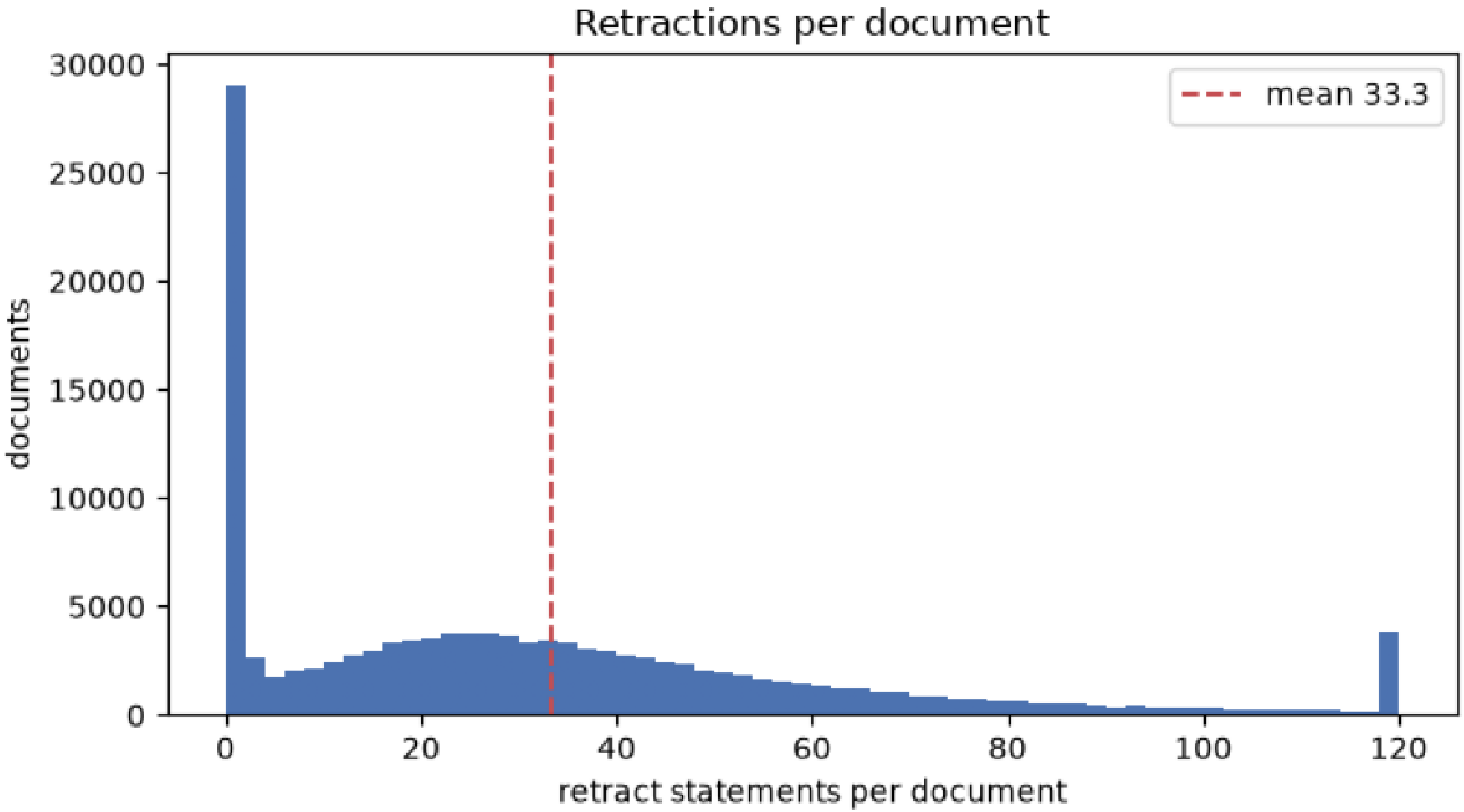
Emit-to-retract distance (mean 17.9, median 9); only 0.2% are immediate. This spread is the long-range self-correction signal the corpus exists to teach.




```
$ make_plots.py --shards 2
```

retracts_per_doc.png

Retractions per document (mean 33.3). 80% of documents contain at least one retraction.



`$ make_plots.py --shards 2`